M4010104 / M4020104: PROGRAMMING IN PYTHON · DIGITAL UNIVERSITY KERALA · CAPSTONE PROJECT SPECIFICATION

Project 15: Indian Grid Daily Power Generation Mix (Hydro, Thermal, Nuclear) & Deficit Analyzer

Effort Budget: 10–12 Hours | Core Stack: Pure Python, NumPy, Pandas, Matplotlib, Seaborn, Plotly | Max Marks: 25 | Instructor: Dr. Sunil TT

1. Executive Overview & Domain Context

India's national electric grid balances daily generation across five regional load dispatch centers (Northern, Western, Southern, Eastern, North-Eastern) by dispatching a dynamic mix of coal-fired Thermal power, Hydroelectric generation, and Nuclear energy against fluctuating peak consumer demand. Seasonal monsoon rains create major swings in hydroelectric availability.

Your team will architect an electric grid transition analytics tool in pure Python that parses daily regional generation logs, cleans negative generation values, calculates the regional Shannon Diversity Index of generation sources in NumPy, models seasonal hydro-thermal trade-offs, and quantifies peak deficit events answering the 12 core questions below.

Primary Public Dataset & Telemetry/Data Schema

The project utilizes the verified public Daily Power Generation in India (2017–2020) on Kaggle (**`https://www.kaggle.com/datasets/navinmundhra/daily-power-generation-in-india-20172020`**). The dataset structure captures:

- **Raw Feature Attributes:** **`Date`**, **`Region`**, **`Thermal_Gen_MU`**, **`Nuclear_Gen_MU`**, **`Hydro_Gen_MU`**, **`Total_Gen_MU`**, **`Peak_Demand_MW`**, **`Deficit_MW`**.
- **Engineered / Target Variables:** **`clean_energy_ratio`** ((Hydro+Nuclear)/Total), **`shannon_diversity_index`**, **`monsoon_hydro_shift`**, **`deficit_severity_tier`**.

2. Complete Project Directory & File Architecture

Each team must organize their repository into a standardized, production-grade layout. All application source code must reside inside a dedicated package (**`src/`**), with separate directories for raw/curated data, interactive development notebooks, visual figures, and formal documentation:

```
indian-grid-generation-mix-analyzer/
├── README.md                          # Repository documentation & GitHub Pages source
├── requirements.txt                   # Dependency specification (numpy, pandas, matplotlib, seaborn, p
├── run_pipeline.py                    # Standalone production CLI application entry point
├── data/
│   ├── raw/
│   │   └── daily_power_generation_india_raw.csv            # Raw uncleaned input dataset (including sim
│   └── processed/
│       ├── clean_daily_power_generation.csv          # Curated, validated clean CSV dataset
│       └── power_grid_audit.log           # Timestamped audit report detailing filtered/rejected record
├── src/
│   ├── __init__.py                    # Python package initialization
│   ├── ingestion.py                   # Defensive line-by-line file reader, range auditor & CSV exporte
│   ├── analyzer.py                    # GridGenerationMixAnalyzer OOP class, NumPy vectorization & Pand
│   └── visualizer.py                  # Matplotlib envelope plots, Seaborn heatmaps & Plotly HTML expor
├── notebooks/
│   └── development_cycle.ipynb        # Development cycle notebook: EDA, trials, tests & unit assertion
└── reports/
    ├── figures/
    │   ├── primary_envelope.png       # Matplotlib confidence envelope & regression overlay
    │   ├── correlation_heatmap.png    # Seaborn annotated feature correlation heatmap
    │   └── distribution_regimes.png   # Seaborn distribution across categorical/domain tiers
```

```
├── interactive_dashboard.html     # Standalone interactive Plotly HTML dashboard with controls
└── Project_Report.pdf             # 3-4 page final technical report
```

3. Dual Development Deliverables: Notebook vs. Standalone Program

To mirror industry software engineering standards, each student group must deliver two distinct, complementary implementations of their solution:

Deliverable A: The Development Cycle Notebook (`notebooks/development_cycle.ipynb`)

The development notebook chronicles your team's engineering journey from raw data exploration to validated model prototypes. It is not merely a final script pasted into cells, but an authentic record of the development cycle:

- **Phase 1: Exploratory Data Ingestion & Diagnostics:** Loading raw data, inspecting missing values, testing boundary rules, and logging rejected records.
- **Phase 2: Mathematical Formulation & Feature Verification:** Developing and validating NumPy array calculations against target benchmarks using `np.allclose()`.
- **Phase 3: OOP Class Architecture & Unit Tests:** Instantiating the `GridGenerationMixAnalyzer` class, executing automated unit assertions (e.g., verifying that passing invalid parameters raises an explicit `ValueError`), and testing edge cases.
- **Phase 4: Statistical Modeling & Regimen Exploration:** Implementing ordinary least-squares regression, rolling-window calculations, and multi-metric Pandas grouping.
- **Phase 5: Visual Prototype Development:** Iteratively designing and refining Matplotlib and Seaborn figures, followed by configuring the interactive Plotly HTML dashboard.

Deliverable B: The Final Standalone Python Program (`run_pipeline.py` & `src/`)

While the notebook documents the exploratory development cycle, your repository must also provide a production-ready, fully executable Python application. The program must run end-to-end directly from the terminal without requiring Jupyter:

- **Command-Line Interface (CLI):** Executed via `python run_pipeline.py --input data/raw/daily_power_generation_india_raw.csv --output-dir data/processed --report-dir reports`.
- **Modular Package Import:** The driver script imports cleanly separated domain logic from `src.ingestion`, `src.analyzer`, and `src.visualizer`.
- **Terminal Feedback & Exit Codes:** The program must print structured progress indicators, log clean/faulty record counts to the console, and return an exit code of `0` upon successful pipeline completion.
- **Automated Artifact Generation:** In a single execution pass, the script reads raw inputs, writes `data/processed/clean_daily_power_generation.csv`, saves `data/processed/power_grid_audit.log`, exports static figures to `reports/figures/`, and generates the standalone `reports/interactive_dashboard.html`.

4. Technical Investigation Requirements & Research Questions (Q1–Q12)

Your team's implementation across the development notebook and standalone program must rigorously answer the following 12 course-aligned engineering questions:

Course Topic & Script	Q#	Concrete Engineering & Analytical Task	Python Libraries & Methods
File I/O & Exceptions (`src/ingestion.py`)	Q1	**Region Integrity & Non-Negative Generation Audit:** Parse raw grid generation CSV rows using try-except. Validate that region names belong to the 5 official grid regions and verify that daily generation values (MU) are strictly non-negative.	open(), try-except (ValueError, KeyError), region checks

Course Topic & Script	Q#	Concrete Engineering & Analytical Task	Python Libraries & Methods
Data Curation & Export (`src/ingestion.py`)	**Q2**	**Curated Grid Generation Export & Audit Log:** Export validated daily records to clean_daily_power_generation.csv while logging mismatched totals (where Thermal+Nuclear+Hydro != Total_Gen) in power_grid_audit.log.	Defensive file writing, dictionary parsing, formatted log reporting
OOP Design (`src/ingestion.py`)	**Q3**	**Grid Generation Mix Analyzer Class:** Design an encapsulated class GridGenerationMixAnalyzer with methods load_from_csv(), compute_shannon_diversity(region='Southern'), and summary_report(). Ensure invalid region raises ValueError.	OOP class, __init__, encapsulation, raise ValueError
NumPy Arrays (`src/analyzer.py`)	**Q4**	**Vectorized Shannon Diversity Index Verification:** Using only raw NumPy arrays of generation shares (p_i), compute the Shannon diversity index (H = -sum(p_i * ln(p_i))) across regions and years. Verify values via np.allclose().	np.log(), np.sum(), np.allclose()
NumPy Masking (`src/analyzer.py`)	**Q5**	**High Clean-Share vs. High Deficit Days Masking:** Flag days where clean energy ratio exceeds 40% in NumPy. Compare against days where peak demand deficit exceeds 1,000 MW using boolean array masks.	np.mean(), np.std(), boolean masks (&, |), Python set
NumPy Regression (`src/analyzer.py`)	**Q6**	**Seasonal Hydro Swing vs. Thermal Output OLS Fit:** Fit an ordinary least-squares model (Thermal_Gen = b0 + b1*Hydro_Gen + b2*Peak_Demand) via np.linalg.lstsq. Quantify the substitution elasticity between hydro and thermal power.	np.column_stack(), np.linalg.lstsq(), matrix product (@), R2
Pandas Series (`src/analyzer.py`)	**Q7**	**Regional Missing Data & Demand Completeness Audit:** Quantify missing daily records and demand gaps across the 5 regions in Pandas. Compare annual generation growth rates across regions.	.groupby('Region').agg(), .isna().sum()
Pandas Groupby (`src/analyzer.py`)	**Q8**	**Region-by-Season Discretization & Pivot Table:** Assign dates to Monsoon (Jun-Sep) vs. Non-Monsoon. Build a Region-by-Season pivot_table(margins=True) displaying mean Hydro share and total generation in MU.	np.where(), .groupby().agg(), .pivot_table(margins=True)
Pandas Reshaping (`src/analyzer.py`)	**Q9**	**Wide-to-Long Generation Source Ranking via Melt:** Reshape generation sources (Thermal, Nuclear, Hydro) into tidy long format via pd.melt() and extract top 5 peak generation days for each energy source.	pd.melt(), .groupby('Energy_Source'), .nlargest(5)
Matplotlib (`src/visualizer.py`)	**Q10**	**Matplotlib Stacked Generation Area Chart:** Plot 3-year daily stacked generation (Hydro, Nuclear, Thermal) for the Southern Region in Matplotlib. Overlay peak demand curves with comprehensive legends.	plt.subplots(), ax.stackplot(), ax.plot(), custom legends
Seaborn (`src/visualizer.py`)	**Q11**	**Seaborn Generation Mix Heatmap & Deficit Boxplots:** Render an annotated correlation heatmap between energy sources and peak demand with sns.heatmap(annot=True). Generate a side-by-side boxplot of deficits across regions.	df.corr(), sns.heatmap(), sns.boxplot(), sns.despine()
Plotly HTML (`src/visualizer.py`)	**Q12**	**Interactive Regional Grid Energy Dashboard:** Build an interactive HTML dashboard (interactive_dashboard.html) displaying regional energy mixes with interactive source toggle buttons, stack plots, and date sliders.	plotly.express.area(), custom hover tooltips, write_html()

5. Collaborative Git Workflow, GitHub Pages & README.md Specification

Best Practices: Collaborative Git Workflow & GitHub Pages Deployment from Day 1

From the very first day, the student team must adopt professional version control and documentation practices. On Day 1, one designated team member initializes the central public GitHub repository, establishes a clear branch protection rule preventing direct pushes to **`main`**, adds all teammates as registered **Contributors** (with write access), and grants read access to the Course Instructor (**`Dr. Sunil TT`**) and teaching assistants. Immediately thereafter, the team must activate **GitHub Pages** (via *Settings → Pages → Build and deployment: Deploy from a branch: main /docs or root*) so that project documentation, interactive architecture diagrams, and the live Plotly dashboard (**`reports/interactive_dashboard.html`**) are publicly accessible and continuously deployed throughout the development lifecycle. All feature development must follow a strict branching model (e.g., **`feature/ingestion`**, **`feature/analysis`**, **`feature/visualization`**): individual team members must write small, atomic, well-commented commits from their own personal GitHub accounts and merge them into **`main`** exclusively through formal **Pull Requests (PRs)** accompanied by peer review. Dumping entire codebases in a single, monolithic end-of-semester commit is strictly prohibited and will result in zero marks for individual Git contribution; maintaining an unbroken, transparent Git commit and PR audit trail is essential for passing the code ownership defense during the live viva-voce examination.

Repository README.md Specification Template

The root **`README.md`** serves as the primary technical landing page and must strictly conform to the following structural format:

````
# Project 15: Indian Grid Daily Power Generation Mix (Hydro, Thermal, Nuclear) & Deficit Analyzer
[![Course](https://img.shields.io/badge/Course-Programming%20in%20Python-blue.svg)](https://duk.ac.in)
[![Python](https://img.shields.io/badge/Python-3.11%2B-brightgreen.svg)](https://python.org)
[![License](https://img.shields.io/badge/License-MIT-yellow.svg)](LICENSE)

## 1. Project Overview & Domain Context
- **Institution:** Digital University Kerala (MTech AI / MSc AI)
- **Course:** M4010104 / M4020104 – Programming in Python | **Instructor:** Dr. Sunil TT
- **Dataset:** Daily Power Generation in India (2017–2020) on Kaggle (https://www.kaggle.com/datasets/na

## 2. System Architecture & Pipeline Design
[Provide an ASCII or Mermaid architecture flow diagram showing raw ingestion -> validation -> analyzer -

## 3. Installation & Environment Setup
```bash
git clone https://github.com/<your-team>/indian-grid-generation-mix-analyzer.git
cd indian-grid-generation-mix-analyzer
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

## 4. Execution Instructions
### A. Running the Standalone Application
```bash
python run_pipeline.py --input data/raw/daily_power_generation_india_raw.csv --output-dir data/processed
```
### B. Running the Development Notebook
```bash
jupyter notebook notebooks/development_cycle.ipynb
```

## 5. Key Analytical & Statistical Findings (Q1–Q12 Summary)
[Provide a concise markdown summary table detailing raw vs. clean counts, regression coefficients, anoma

## 6. Interactive Telemetry Dashboard
Explore the live interactive Plotly visualization hosted on GitHub Pages:
🔗 [Live Project Dashboard](https://<your-team>.github.io/indian-grid-generation-mix-analyzer/interactiv
````

```
## 7. Contributor Git Commit & Pull Request Log Table
| Contributor Name | GitHub Handle | Assigned Module / Tasks | Verified Commit Hash Range | Merged Pull
| :--- | :--- | :--- | :--- | :--- |
| [Student Full Name] | `@github_handle` | Modular Architecture (`src/ingestion.py`, `src/analyzer.py`,
```

6. Evaluation Rubric & Mandatory Academic Integrity Governance

Projects are evaluated on a 25-mark scale based on code quality, mathematical correctness, visualization depth, git collaboration discipline, and live technical defense:

Assessment Criterion	Basis of Evaluation & Deliverable Verification	Max Marks
1. Data Curation & File I/O Pipeline (Q1–Q2)	Robust line-by-line file reading with defensive **`try-except`** handling, physical/domain bounds validation, clean CSV output, and structured **`power_grid_audit.log`** reporting.	**5 Marks**
2. OOP, NumPy & Pandas Analytics (Q3–Q9)	Modular **`GridGenerationMixAnalyzer`** class architecture, vectorized NumPy feature re-derivations (verified via **`np.allclose`**), regression modeling, and Pandas rolling/groupby/melt operations.	**6 Marks**
3. Multi-Library Visualizations (Q10–Q12)	Publication-quality Matplotlib confidence envelope/trend plot, Seaborn correlation heatmap and distribution violin plot, and standalone interactive Plotly HTML dashboard.	**5 Marks**
4. Software Packaging, GitHub & Documentation	Clean directory layout, comprehensive **`README.md`**, working GitHub Pages site, **`development_cycle.ipynb`** notebook, standalone **`run_pipeline.py`**, and balanced individual Git commits/PRs.	**4 Marks**
5. Live Presentation & Code Viva-Voce Defense	Technical walkthrough and individual live viva defense where each student must explain or modify lines of code in real time without external assistance.	**5 Marks**

> **STRICT ACADEMIC INTEGRITY & ZERO-TOLERANCE AI POLICY (0 MARKS PENALTY):** All submitted Python source code (**`.py`**, **`.ipynb`**) and written documentation will be screened through an automated **AI-Code & Plagiarism Detection Suite**. If AI-generated code or synthetic documentation is detected at any checkpoint or final submission, **0 Marks will be awarded immediately** for the entire project. Each student must prove authentic individual code ownership during the live viva-voce examination — *Course Instructor: Dr. Sunil TT*.

7. Milestone Schedule, Submission Checklist & Viva Defense Prompts

To ensure systematic progress across the semester, teams must adhere to three project checkpoints and prepare for on-the-spot live code modification challenges during the viva-voce examination:

Milestone Stage	Scheduled Window	Required Deliverables & Verification Milestones
Checkpoint 1 (Data Ingestion)	Week 11 Lab Session	Repository initialized with contributors and GitHub Pages active. **`src/ingestion.py`** operational with line-by-line filtering, **`clean_daily_power_generation.csv`** exported, and **`power_grid_audit.log`** generated.
Checkpoint 2 (Analytics & OOP)	Week 13 Lab Session	**`src/analyzer.py`** complete with **`GridGenerationMixAnalyzer`** class, NumPy feature checks (Q4), outlier masks (Q5), regression modeling (Q6), and exploratory notebook sections.

Milestone Stage	Scheduled Window	Required Deliverables & Verification Milestones
Final Submission & Viva	Week 15 Project Review	Full package operational: standalone **`run_pipeline.py`**, polished **`development_cycle.ipynb`**, interactive **`interactive_dashboard.html`** on GitHub Pages, 3–4 page PDF report, and live code defense.

Representative Live Viva-Voce Code Modification Challenge

During the individual code viva-voce, the instructor will ask students to execute an immediate 2-minute code modification on their working repository to demonstrate genuine authorship:

- **Challenge Prompt:** *"In src/analyzer.py, compute the Southern Region's hydro generation share in August (monsoon peak) vs. May (pre-monsoon summer). Re-run your module and print the percentage swing."*